

OOP Concepts in C++

1. Learn Key Concepts of Object-Oriented Programming Paradigm

Object-oriented programming (OOP) is a paradigm that structures software around objects that represent real-world entities.

It focuses on modeling systems using data (attributes) and behaviors (methods).

The four fundamental principles of OOP are:

1. Encapsulation:

- Bundling data and methods inside a class.
- Restricts access to data using access modifiers.

Example:

```
class Car {  
    private:  
        int speed;  
    public:  
        void setSpeed(int s) { speed = s; }  
};
```

2. Abstraction:

- Showing only essential features and hiding internal details.

Example:

Driving a car requires pressing the accelerator without knowing how the engine works internally.

3. Inheritance:

- A child class reuses and extends behavior of a base class.

Example:

```
class Vehicle {};  
  
class Car : public Vehicle {};
```

4. Polymorphism:

- One interface, many implementations.
- Achieved through method overloading or overriding.

2. Apply OOP Principles to Implement C++ Programs

In C++, you apply OOP by creating classes that represent objects and using them in `main()`.

Example:

```
class Student {  
  
private:  
  
    string name;  
  
    int age;  
  
public:  
  
    void setData(string n, int a) {  
  
        name = n;  
  
        age = a;  
  
    }  
}
```

```
void print() {  
    cout << "Name: " << name << ", Age: " << age << endl;  
}  
};  
  
int main() {  
    Student s;  
    s.setData("Ahmed", 20);  
    s.print();  
}
```

3. Utilize Default Arguments

Default arguments allow functions to be called without supplying all parameters.

Example:

```
int add(int a, int b = 10, int c = 5) {  
    return a + b + c;  
}  
  
int main() {  
    cout << add(5) << endl;    // Uses default values: 5+10+5 = 20  
    cout << add(5, 2) << endl; // Uses default c: 5+2+5 = 12  
    cout << add(5, 2, 3) << endl; // 5+2+3 = 10  
}
```

4. Apply Function Overloading

Function overloading allows multiple functions with same name but different parameter lists.

Example:

```
int area(int side) { return side * side; }
```

```
int area(int l, int w) { return l * w; }
```

```
double area(double radius) { return 3.14 * radius * radius; }
```

Now:

```
area(5);    // square
```

```
area(4, 6); // rectangle
```

```
area(2.5);  // circle
```

5. Explain Constructors and Destructors

Constructor:

- Special function called automatically when object is created.
- Used to initialize values.
- Has same name as class.

Destructor:

- Called when the object is destroyed.
- Starts with '~'.
- Used to release memory.

Example:

```
class Example {  
public:  
    Example() { cout << "Constructor called"; }  
    ~Example() { cout << "Destructor called"; }  
};
```

6. Apply Stack Operations (Push and Pop)

A stack is a LIFO (Last In First Out) data structure.

Operations:

- push() – insert
- pop() – remove top element

Example using STL:

```
stack<int> s;  
s.push(10);  
s.push(20);  
s.pop();
```

Custom stack:

```
class Stack {  
private:  
    int arr[5];  
    int top;
```

public:

```
Stack() { top = -1; }
```

```
void push(int x) { arr[++top] = x; }
```

```
int pop() { return arr[top--]; }
```

```
};
```

7. Define Static Members

Static members:

- Shared among all objects.
- Belong to the class, not object.
- Must be defined outside class.

Example:

```
class Test {
```

```
public:
```

```
    static int count;
```

```
    Test() { count++; }
```

```
};
```

```
int Test::count = 0;
```

```
int main() {
```

```
    Test a, b, c;
```

```
    cout << Test::count; // 3
```

```
}
```

8. Friend Function vs Member Function

Member function:

- Belongs to class and can access all members.

Friend function:

- Not part of class, but has access to private members.
- Useful for external operator overloading.

Example:

```
class Box {  
private:  
    int width;  
public:  
    Box(int w) : width(w) {}  
    friend void print(Box b);  
};  
  
void print(Box b) {  
    cout << b.width;  
}
```

9. Learn When and How to Use a Copy Constructor

A copy constructor creates a new object from an existing one.

Used when:

- Passing objects by value
- Returning objects
- Deep copying dynamic memory

Example:

```
class A {  
public:  
    int *ptr;  
    A(int x) {  
        ptr = new int(x);  
    }  
    A(const A &obj) {    // Copy constructor  
        ptr = new int(*obj.ptr);  
    }  
};
```

10. Implement Assignment Operator (=)

Used to assign one object to another after creation.

Example:

```
Stack& operator=(const Stack &s) {  
    if(this != &s) {  
        delete[] arr;  
        size = s.size;  
        arr = new int[size];  
    }
```



```
}  
    return *this;  
}
```

11. Use Operator Overloading

Operator overloading allows using operators with objects.

Example (+ overload):

```
class Number {  
public:  
    int value;  
    Number(int v) : value(v) {}  
    Number operator+(Number n) {  
        return Number(value + n.value);  
    }  
};
```

```
Number a(5), b(3);
```

```
Number c = a + b; // 8
```

12. Association, Aggregation, and Composition

Association:

- Loose relationship (Teacher teaches Student).

Aggregation:

- Whole-part, but part can exist independently.
- Example: Department has Teachers.

Composition:

- Strong relationship; part cannot exist without whole.
- Example: Human has a Heart.

13. Explain Polymorphism

Two types:

1. Compile-time Polymorphism:

- Function overloading
- Operator overloading

2. Run-time Polymorphism:

- Achieved using virtual functions

Example:

```
class Base {  
public:  
    virtual void show() { cout << "Base"; }  
};
```

```
class Derived : public Base {  
public:
```

```
void show() { cout << "Derived"; }  
};
```

```
Base *b = new Derived();  
b->show(); // "Derived"
```

14. Access Specifiers

public:

- Accessible anywhere.

private:

- Accessible only within class.

protected:

- Accessible in class and derived classes.

Example:

```
class A {
```

```
private:
```

```
    int a;
```

```
protected:
```

```
    int b;
```

```
public:
```

```
    int c;
```

```
};
```

15. Design and Implement Multiple Inheritance

Example:

```
class A { };
```

```
class B { };
```

```
class C : public A, public B { };
```

16. Compile-Time vs Run-Time Binding

Compile-time Binding:

- Done by the compiler.
- Examples: overloading.

Runtime Binding:

- Achieved using virtual functions.
- Decided at program execution time.

17. Implement Template Classes

Templates allow generic programming.

Example:

```
template <class T>
```

```
class Calculator {
```

```
public:
```

```
    T add(T a, T b) { return a + b; }
```

```
};
```

```
Calculator<int> c1;
```

```
Calculator<double> c2;
```